

F# is better than C#



Better Abstractions

Better Semantics

Better Defaults

*"Point of view is worth 80 IQ
points"*

Alan Kay

What is F#?

General purpose language on .NET
designed by Don Syme,
from Microsoft Research

What is F#?

- Strongly, statically typed, open sourced and cross platform language with deep inference
- 1.0 appeared in 2005 (older than: Go, Rust, Kotlin, TypeScript...)
- Heavily influenced by OCaml, Haskell & C#
- Multi-paradigm language: functional (by default), imperative and object-oriented
- Fully supported by .NET. Compiles to JavaScript, TypeScript, Python, Rust...

F# in the Real World

Finance & Trading

Credit Suisse, G-Research, Millenium, Jet.com

Big Tech

Microsoft, Walmart Labs, Trainline

Energy & Industrial

Tachyus (Oil & Gas), GreenPowerMonitor, AutoStore

Healthcare & Biotech

Amyris, LetsGetChecked

Where correctness matters, F# delivers.

The Indisputable Fact

C# keeps borrowing from F#

- Type inference (F# 2005 → C# var* 2007)
- Async workflows (F# 2007 → C# async/await 2012)
- Tuples (F# 2005 → C# 2017)
- Init-only properties (F# 2005 → C# 2020)
- Collection expressions (F# 2005 → C# 2023)
- IEnumerable (F# 2007 → C# 2019)
- ...and so on

What else is F# right about?

Better Abstractions

Discriminated Unions vs Class Hierarchies

- Less structure, less boilerplate
- Compiler-guaranteed exhaustiveness
- Missing cases caught at compile time
- One paradigm, not inheritance + interfaces + sealed + switch

Discriminated Unions

Classic C#

```
public abstract class Command { }
public class Start : Command { }
public class Stop : Command { }
public class SetValue : Command
{
    public int Value { get; }
    public SetValue(int value) => Value = value;
}

// The REAL OO way? 1 visitor interface + 1 abstract class + 1 class per case + 1 handler
public string Handle(Command cmd)
{
    if (cmd is Start) return "Starting...";
    if (cmd is Stop) return "Stopping...";
    if (cmd is SetValue sv) return $"Set {sv.Value}";
    throw new InvalidOperationException();
}
```

Discriminated Unions

Modern C#

```
public abstract record Command;  
public record Start : Command;  
public record Stop : Command;  
public record SetValue(int Value) : Command;  
  
string Handle(Command cmd) => cmd switch  
{  
    Start => "Starting...",  
    Stop => "Stopping...",  
    SetValue sv => $"Set {sv.Value}",  
    _ => throw new InvalidOperationException()  
};
```

`_ => throw` required. No compile-time exhaustiveness.

Discriminated Unions

F#

```
type Command =  
    | Start  
    | Stop  
    | SetValue of int  
  
let handle = function  
    | Start -> "Starting..."  
    | Stop -> "Stopping..."  
    | SetValue v -> sprintf "Set %d" v
```

Exhaustiveness checked at compile time.

Pattern Matching

Classic C#

```
// The REAL OO way? 1 visitor interface + 1 abstract class + 1 class per case + 1 handler
public string Handle(Command cmd)
{
    if (cmd is Start || cmd is Stop || cmd is Pause)
        return "Simple";
    if (cmd is SetValue sv)
    {
        if (sv.Value == 0) return "Reset";
        if (sv.Value < 0) return "Invalid";
        return $"Set {sv.Value}";
    }
    if (cmd is SetRange sr)
    {
        if (sr.Min == 0 && sr.Max == 100) return "Default";
        return $"{sr.Min}-{sr.Max}";
    }
    throw new InvalidOperationException();
}
```

Pattern Matching

Modern C#

```
string Handle(Command cmd) => cmd switch
{
    Start or Stop or Pause => "Simple",
    SetValue { Value: 0 } => "Reset",
    SetValue { Value: < 0 } => "Invalid",
    SetValue sv => $"Set {sv.Value}",
    SetRange { Min: 0, Max: 100 } => "Default",
    SetRange { Min: var min, Max: var max } => $"{min}-{max}",
    _ => throw new InvalidOperationException()
};
```

Verbose property patterns: { Min: var min, Max: var max }

Pattern Matching

F#

```
let handle = function
    | Start | Stop | Pause -> "Simple"
    | SetValue 0 -> "Reset"
    | SetValue v when v < 0 -> "Invalid"
    | SetValue v -> sprintf "Set %d" v
    | SetRange (0, 100) -> "Default"
    | SetRange (min, max) -> sprintf "%d-%d" min max
```

6 lines. Or patterns, guards, constants, tuple deconstruction.

Better Semantics

Immutable

by default

Explicit when you need
mutation

Expression

based

Everything returns a value

No nulls

from day one

Not retrofitted annotations

SOLID exists because OO by itself does not compose well. FP does.

Immutability & Expressions

Modern C#

```
int GetDiscount(int customerId)
{
    var customer = TryFindCustomer(customerId);
    if (customer?.IsActive != true) return 0;

    customer.Tier = ApplyPromotion(customer.Tier); // mutates original!
    return TierToDiscount(customer.Tier);
}
```

Null ceremony + silent mutation = bugs

Immutability & Expressions

F#

```
let getDiscount customerId =  
    customerId  
    |> tryFindCustomer  
    |> Option.filter isActive  
    |> Option.map applyPromotion    // returns NEW customer  
    |> Option.map tierToDiscount  
    |> Option.defaultValue 0
```

Immutable. Expression-based. No null checks. Just flow.

Pipe Operator & Composition

Modern C#

```
// No pipe - nested calls or temp variables
var result = (int)Math.Round(Math.Sqrt((float)42));

// Temp variable chain
var parsed = Parse(data);           // string -> Order
var validated = Validate(parsed);   // Order -> Order
var transformed = Transform(validated); // Order -> Invoice
var result = Save(transformed);     // Invoice -> Result

// Composition? Build it yourself.
Func<Order, Invoice> processOrder = order =>
    FormatInvoice(
        CalculateTax("FR",
            ApplyDiscount(10,
                ValidateOrder(order))));
```

Pipe Operator & Composition

F#

```
// Pipe works on anything
42 |> float |> sqrt |> round |> int // int -> float -> float -> float -> int

// Data transformation
let result =
    data
    |> parse      // string -> Order
    |> validate   // Order -> Order
    |> transform  // Order -> Invoice
    |> save       // Invoice -> Result

// Composition + partial application
let processOrder =      // Order -> Invoice
    validateOrder
    >> applyDiscount 10 // partially applied
    >> calculateTax "FR" // partially applied
    >> formatInvoice
```

Better Defaults

Every C# best practice guide is describing F#.

C# *"Prefer immutability"*

:
F# **let is immutable by default**
:

C# *"Avoid null, use nullable reference types"*

:
F# **Option<T> since 2005. No null.**
:

C# *"Favor composition over inheritance"*

:
F# **Functions + pipes. No classes needed.**
:

C# *"Write pure functions, avoid side effects"*

:
F# **Pure by default. Side effects are explicit.**
.

In C#, you learn what NOT to do. In F#, you just write code.

The Ecosystem

It's all .NET.

The only difference is the code you write.

- Same NuGet packages
- Same runtime
- Same deployment
- Full C# interop



the better language



the market choice

Thank you

Q&A

Let's debate.